Introducción al Proyecto — Predicción de Churn en E-commerce (Olist)

David Alan Delgado Zarco
Octavio Esquivel Alvarez del Castillo
John Williams Morrill
Jason Rosher Morrill

March 10, 2026

Contents

Distribución del tiempo **2**

1 Segmento 1 — Introducción **3**

2 Segmento 2 — Dataset y Problema **3**

3 Segmento 3 — Features y Partición Temporal **3**

4 Segmento 4 — Modelos y Evaluación **4**

5 Segmento 5 — MLflow en AWS EC2 **4**

6 Segmento 6 — Tablero Interactivo y Demo **4**

7 Segmento 7 — Conclusiones **5**

1 Motivación y pregunta de negocio

El *churn*, o abandono de clientes, representa uno de los mayores desafíos operativos en el comercio electrónico. Retener a un cliente existente es significativamente más económico que adquirir uno nuevo; sin embargo, muchas plataformas carecen de mecanismos automáticos para identificar, antes de que ocurra, qué clientes están en riesgo de no volver a comprar.

Este proyecto aborda esa problemática sobre el **dataset público de Olist**, un marketplace brasileño con datos reales de órdenes, clientes, pagos, reseñas e ítems. La pregunta central que guía el trabajo es: **¿es posible anticipar qué clientes no volverán a comprar en los próximos 90 días, a partir de su historial transaccional y de experiencia?**

Para responderla se construyó un pipeline completo de Machine Learning: desde la definición rigurosa del label hasta el despliegue de un modelo en producción en AWS, incluyendo un tablero interactivo de predicción individual.

2 Datos y definición del label

Se utilizaron cinco archivos del dataset de Olist: `orders`, `customers`, `payments`, `reviews` e `items`. Para evitar la duplicación de registros provocada por relaciones 1:N, cada tabla auxiliar se agrega por `order_id` antes de unirse a la tabla de órdenes.

El aspecto más crítico del diseño fue construir el label de churn sin introducir *data leakage*. La solución adoptada es una **arquitectura de ventana temporal**: se fija una **fecha de snapshot** y se observa si el cliente realiza alguna compra en la **ventana futura** de $W = 90$ días siguiente. Formalmente:

$$\text{churn} = 1 \iff \text{el cliente no compra en } (\textit{snapshot},\ \textit{snapshot} + 90 \text{ días}]$$

Un cliente se etiqueta como churn = 0 (no-churn / recompra) si sí compra dentro de esa ventana. Adicionalmente, para excluir clientes históricamente inactivos que sesgarían el modelo, la población se limita a aquellos con al menos una compra dentro de un *lookback* de 180 días previos al snapshot.

3 Ingeniería de features y partición temporal

Las variables se construyen a nivel de `customer_unique_id`, consolidando todas las órdenes anteriores al snapshot en tres grupos:

- **RFM:** *Recency* (días desde la última compra), *Frequency* (número de órdenes únicas) y *Monetary* (suma total de pagos).
- **Experiencia y logística:** promedio del puntaje de review, días de entrega, días de retraso frente a la fecha estimada, número de ítems por orden, suma promedio de precio y de flete.
- **Contexto geográfico:** estado del cliente en Brasil (`customer_state`), tratado como variable categórica con codificación *one-hot*.

La partición train/test se realiza de forma **temporal** —no aleatoria— ordenando los clientes por su última compra previa al snapshot y asignando el 80% más antiguo a entrenamiento y el 20% más reciente a prueba. Esto reproduce un escenario realista de generalización y elimina cualquier filtración de información futura.

4 Modelos y evaluación

Se entrenaron y compararon dos familias de modelos, ambas con corrección por desbalance de clases:

- **Regresión Logística** con `class_weight=balanced` y regularización L_2.
- **Random Forest** con `class_weight=balanced_subsample` y `n_estimators=400`.

Dado el fuerte desbalance del dataset —la mayoría de los clientes de Olist son compradores únicos que no regresan—, la métrica principal adoptada es el **PR-AUC para la clase no-churn** (`pr_auc_nochurn_0`), que evalúa la capacidad del modelo de identificar correctamente la recompra. El accuracy convencional resulta engañoso en este contexto: un clasificador trivial que prediga siempre churn alcanza alta exactitud pero fracasa completamente en detectar la clase de interés.

Un segundo aspecto clave fue el **ajuste del umbral de decisión**. Mediante un barrido sobre 99 valores entre 0 y 1, se identificó el umbral que maximiza el F_1 para no-churn. El valor resultante fue $\hat{\tau} = 0.12$, considerablemente inferior al 0.5 convencional, lo cual refleja la rareza del evento de recompra en la población.

El modelo seleccionado fue la **Regresión Logística con** $C = 0.1$ y umbral operativo 0.12, registrado en MLflow como `logreg_C0.1_thr012` por presentar el mayor `pr_auc_nochurn_0` entre todos los runs comparados.

5 Gestión de experimentos con MLflow en AWS EC2

Todo el ciclo experimental se gestionó con **MLflow Tracking Server**, desplegado en una instancia **EC2 Ubuntu** de AWS. El servidor almacena los parámetros, métricas y artefactos —modelo serializado, matrices de confusión, curvas Precision-Recall— de cada experimento, permitiendo su comparación visual en la interfaz web accesible por IP pública.

Esta infraestructura garantiza la reproducibilidad de los experimentos y la trazabilidad completa del proceso de selección del modelo, tal como se esperaría en un entorno de producción real.

6 Tablero de predicción individual

El proyecto culmina con el despliegue de un **tablero interactivo** accesible desde el navegador. Su arquitectura se compone de tres capas:

- **Frontend React/Vite:** captura las variables del cliente a través de un formulario lateral y muestra el resultado de la predicción.
- **API Express (Node.js):** recibe la solicitud en el endpoint `POST /api/predict` y orquesta la ejecución del proceso de inferencia.
- **Script de inferencia Python:** carga el modelo `best_model_logreg.joblib` y ejecuta `predict_proba`, devolviendo la probabilidad de churn como JSON.

El flujo de datos de extremo a extremo es:

$$\text{Usuario} \xrightarrow{\text{POST}} \text{React/Vite} \rightarrow \text{Express} \xrightarrow{\texttt{spawn}} \text{Python} \rightarrow \texttt{joblib} \rightarrow \texttt{churnProbability}$$

El tablero muestra la probabilidad de churn en porcentaje junto con el nivel de riesgo (bajo, medio o alto) y, de forma destacada, un panel de **contribución de features** que desglosa qué variables impulsaron la predicción hacia churn o hacia retención. Esta interpretabilidad —derivada directamente de los coeficientes de la regresión logística— convierte el resultado en una herramienta accionable para equipos de retención.

7 Conclusiones

El proyecto demuestra que es posible construir un sistema de predicción de churn funcional y desplegado en producción con un stack accesible: datos abiertos de e-commerce, pipeline reproducible con MLflow, modelo lineal interpretable y tablero en la nube.

Los aprendizajes técnicos centrales son:

- El diseño correcto del label mediante ventana temporal es indispensable para evitar *data leakage* y obtener estimaciones de generalización válidas.
- En datasets desbalanceados, el ajuste del umbral de decisión tiene un impacto mayor que la elección del algoritmo.
- MLflow proporciona trazabilidad y reproducibilidad sin sobrecarga operativa significativa.

Como trabajo futuro se identifican: incorporar señales de comportamiento por categoría de producto, explorar modelos de gradient boosting calibrados, y añadir monitoreo de deriva del modelo (*data drift*) en producción.